CLAUDE CODE 실습

시승 예약 시스템 구현

프롬프트만으로 만드는 Next.js와 Supabase 예약 앱

폴스타 3 · 폴스타 4 · 폴스타 5 시승 예약

누가 어디에서 무엇을 입력하고, 무엇이 나오며, 무엇을 확인하는가.
모든 단계를 이 다섯 칸으로 따라 한다.

핵심

이 실습은 세 가지 원칙으로 움직인다

코드는 한 줄도 직접 쓰지 않는다는 전제로 구성했다

1

코드는 Claude Code가 쓴다

수강생은 프롬프트 붙여넣기, 계정 승인, 화면 확인만 한다

2

기준 문서가 프롬프트를 짧게 만든다

requirements.md와 data 폴더를 읽혀 규칙을 매번 다시 말하지 않는다

3

중복 예약은 화면이 아니라 DB가 막는다

트랜잭션과 partial unique index로 막고, 7단계에서 직접 깨뜨려 본다

막히면 고치려 하지 말고, 본 것을 문장으로 써서 Claude Code에게 돌려준다

완성하면 고객은 휴대폰으로 시승을 예약한다

왼쪽은 PC 첫 화면, 오른쪽은 휴대폰의 3단계 날짜와 시간 선택이다 (화면 예시)

localhost:3000

DEMO · 실제로 저장되지 않음

원하는 차를 골라 시승하세요

교육용 전시장 A · 1시간 시승 · 내일부터 30일간



폴스타 3

전기 SUV

이 차로 시승 예약



폴스타 4

전기 SUV 쿠페

이 차로 시승 예약



폴스타 5

전기 4도어 GT

이 차로 시승 예약

3 / 5 단계

날짜와 시간

폴스타 4 · 교육용 전시장 A

1	2	3	4	5	6	7
8	9	10	11	12	13	14

10:00 | ~~11:00~~ | 12:00

13:00 | **14:00** | ~~15:00~~

16:00 | 17:00

다음 · 고객 정보

이전 단계

시승 차종은 폴스타 3, 4, 5 세 가지다

이미지는 실습 키트 assets/images에 들어 있고 data/models.csv와 연결된다



폴스타 3

전기 SUV

polestar-3.jpg



폴스타 4

전기 SUV 쿠페

polestar-4.jpg 외 1장



폴스타 5

전기 4도어 GT

polestar-5-front.jpg 외 2장

사이트에 함께 쓰인 다른 브랜드 사진은 시승 차종이 아니므로 키트에서 뺐다

예약 앱 하나가 고객, 직원, DB를 잇는다

Next.js는 화면과 서버 API를, Supabase는 예약 DB와 직원 로그인을 맡는다

고객
휴대폰과 PC

직원
관리자 화면

기존 아주 사이트
'시승 예약' 버튼

→

Next.js 예약 앱
Vercel에 배포

/reserve · 고객 예약 5단계

/admin · 확정과 취소

서버 API · 입력 재검증

비밀키는 서버에서만 쓴다

⇄

Supabase

PostgreSQL · 예약 DB
중복 방지와 RLS

Auth · 직원 로그인

Claude Code
코드 작성 · GitHub 저장

고객 접수와 직원 확정은 서로 다른 단계다

접수(pending)만으로는 확정되지 않는다. 직원이 고객과 통화한 뒤 확정한다

고객 · 예약 5단계

① 차종 — 폴스타 3·4·5 → ② 전시장 — 전시장 A → ③ 날짜·시간 — 내일~30일 → ④ 정보·동의 — 이름·휴대폰 → ⑤ 확인 — 접수 완료

↓

상태 pending (접수) · 슬롯 점유

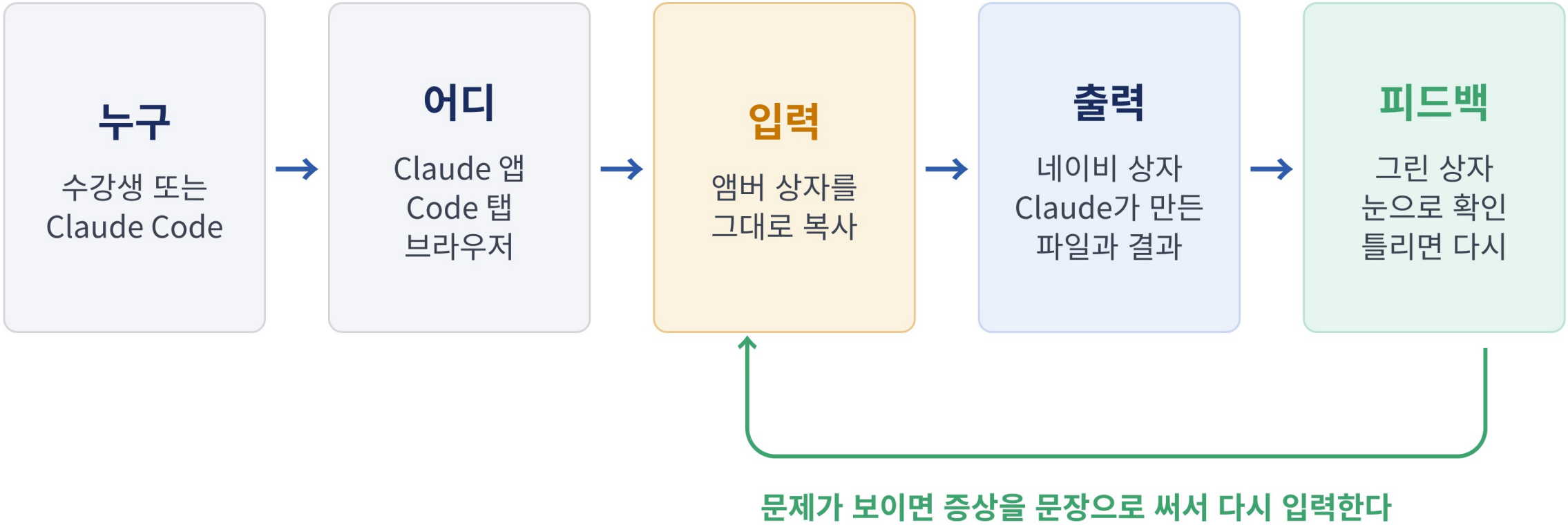
직원 목록에 바로 나타남 ←

직원 · 관리자 화면

접수 확인 — 매일 목록 → 고객 통화 — 일정 확인 → 확정 — confirmed → 시승 진행 — 전시장 → 방문 처리 — completed

모든 단계는 다섯 칸으로 읽는다

앰버는 내가 넣는 것, 네이비는 Claude가 내놓는 것, 그린은 내가 확인할 것이다



열 단계를 거치면 배포된 예약 앱이 남는다

각 칸 아래는 그 단계가 끝났을 때 손에 남는 산출물이다

직접 조작

0

준비

Claude Code 실행

1

규칙

CLAUDE.md

2

화면

localhost 화면

직접 조작

3

Supabase

가입 후 자동 연결

4

DB

테이블 8개

5

저장

예약 1건 저장

6

관리자

확정과 취소

7

테스트

test-report

8

배포

미리보기 주소

9

버튼 연결

기존 사이트

앰버 표시는 앱 설치와 Supabase 가입처럼 사람만 할 수 있는 단계. 나머지는 프롬프트만 붙여 넣는다

운영 규칙은 실습용 가정으로 먼저 고정한다

실제 운영시간과 차량 보유는 담당자가 확정한다. 값은 data/booking_rules.json에 있다

항목	실습 기본값
차종	폴스타 3, 4, 5
전시장	교육용 전시장 A 한 곳
시승차	차종마다 1대
운영시간	10~18시, 1시간 단위
예약 기간	내일부터 30일 뒤까지

항목	실습 기본값
최초 상태	pending (접수)
슬롯 점유	pending과 confirmed
고객 입력	이름, 휴대폰, 필수 동의
수집 금지	주민번호, 면허증 사진
시간대	Asia/Seoul 기준

접수 화면 문구 — "예약이 접수되었습니다. 담당자 확인 후 확정 안내드립니다."

실습 키트가 프롬프트를 짧게 만든다

Claude Code는 매 단계 docs와 data를 읽고 일한다. 파일: AJU_TestDrive_LabKit_II.zip

파일	내용	쓰는 단계
docs/requirements.md	요구사항 명세 1~10장	1~7
docs/test_scenarios.md	테스트 시나리오 14개	7
docs/privacy_consent_draft.md	교육용 동의 문구 초안	5
data/models.csv 외 CSV 3개	차종, 전시장, 시승차, 상태 전이	2, 4
data/booking_rules.json	운영시간, 기간, 화면 문구, rate limit	2, 5
data/test_customers.csv	가짜 테스트 고객 10명	5, 7
assets/images	폴스타 이미지 6장	2
prompts/PROMPTS_KR.md	이 덱의 프롬프트 원문 전체	전체

01

0단계 · 준비

데스크톱 앱에서 Claude Code를 연다

앱을 설치하고 작업 폴더를 연다. 이후는 모두 Code 탭에서 한다

Code 탭 화면은 다섯 곳만 알면 된다

개념도다. 실제 배치는 앱 버전에 따라 조금 다르다



순서

① 왼쪽 **Code** 탭
② 입력창 아래 **Local**
③ 폴더 → **Open folder...**
④ 권한 모드 확인
⑤ 프롬프트 붙여넣기

터미널 창은 Ctrl+`
미리보기는 Browser 창
(개념도, 실제 배치는
버전마다 다르다)

데스크톱 앱의 Code 탭에서 Claude Code를 연다

터미널 없이 쓰는 Claude Code다. Pro 이상의 유료 계정이 필요하다

누구 수강생 (직접)

어디 Claude 데스크톱 앱

0 준비 · 1 규칙 · 2 화면 · 3 연결 · 4 DB · 5 저장 · 6 관리자 · 7 테스트 · 8 배포 · 9 버튼

[입력] 직접 하는 일

① claude.com/download에서 Windows용 설치 파일을 받아 실행한다

② 앱을 열고 Claude 계정으로 로그인한다

③ 문서 폴더에 aju-testdrive 폴더를 만든다

④ 왼쪽 위 Code 탭을 누르고 입력창 아래에서 Local을 고른다

⑤ 폴더 칸의 Open folder...에서 aju-testdrive를 고른다

⑥ 권한 모드는 Accept edits로 둔다 (수정은 자동, 명령은 확인)

[출력] 결과 · Code 탭

프롬프트 입력창이 열림
입력창 아래에 Local과
aju-testdrive가 표시됨

피드백 · 이것을 확인한다

- 폴더 이름이 보이는가
- Git 오류는 부록 A-1로
- 유료 계정으로 로그인했나

키트 풀기와 도구 설치도 Claude Code가 한다

키트 zip을 다운로드 폴더에 받아 둔 뒤 시작한다

누구 Claude Code (붙여넣기)　**어디** Claude 앱 Code 탭

[입력] 프롬프트 · Claude 앱 · Code 탭 입력창

```
다운로드 폴더의 AJU_TestDrive_LabKit_II.zip을
현재 폴더에 풀어 주세요. 기존 파일은 덮어쓰지 마세요.
README.md를 읽고 폴더 구조를 표로 보여 주세요.

node, npm, git 버전을 확인하세요.
없거나 너무 오래된 것이 있으면 winget으로 직접
설치하고, 설치 후 다시 확인해 결과를 알려 주세요.
앱 코드는 아직 만들지 마세요.
```

[출력] Claude 응답

```
| 폴더           | 내용              |
| docs           | 요구사항, 테스트 |
| data           | CSV 5개, JSON 1개 |
| assets/images | 폴스타 사진 6장   |
| prompts        | 프롬프트 원문     |
node, npm, git 버전 보고
```

피드백 · 이것을 확인한다

- 폴더 네 개가 보이는가
- Node와 Git이 설치됐나
- 명령 승인은 읽고 허용한다

02

1~2단계 · 화면

규칙을 세우고 화면부터 만든다

DB 없이 mock 데이터로 고객 흐름을 먼저 검증한다

첫 프롬프트는 코드가 아니라 규칙을 만든다

CLAUDE.md는 Claude Code가 세션마다 먼저 읽는 작업 수칙 파일이다

누구 Claude Code (붙여넣기)　**어디** Claude 앱 Code 탭

[입력] 프롬프트 · Claude 앱 · Code 탭 입력창

```
docs/requirements.md와 data 폴더를 읽으세요.
이를 기준으로 CLAUDE.md와 docs/plan.md를 만드세요.
앱 코드는 아직 작성하지 마세요.

CLAUDE.md에 넣을 작업 규칙:
- 한 단계씩 구현하고, 끝나면 직접 실행해 확인한 뒤
  내가 눈으로 볼 항목을 알려 주고 멈춘다
- 비밀값을 대화, 로그, Git에 출력하지 않는다
- 실행하지 않은 검사를 통과했다고 쓰지 않는다
- 단계가 끝날 때마다 Git 커밋을 남긴다

plan.md에는 고객 여정, 관리자 여정, 데이터 구조,
단계별 완료 조건을 넣으세요.
Git 저장소가 없으면 git init 후 첫 커밋을 하세요.
```

[출력] Claude가 생성

```
CLAUDE.md
  작업 규칙 4개와 기술 스택
docs/plan.md
  고객 여정, 관리자 여정
  데이터 구조, 단계별 완료 조건
Git 첫 커밋 1개
```

피드백 · 이것을 확인한다

- plan.md를 열어 본다
- 운영시간이 10~18시인가
- 다르면 2장대로 고치게 한다

mock 데이터로 고객 화면 전체를 먼저 띄운다

DB 연결 전이므로 화면 상단에 DEMO 띠가 붙는다

누구 Claude Code (붙여넣기) **어디** Claude 앱 Code 탭

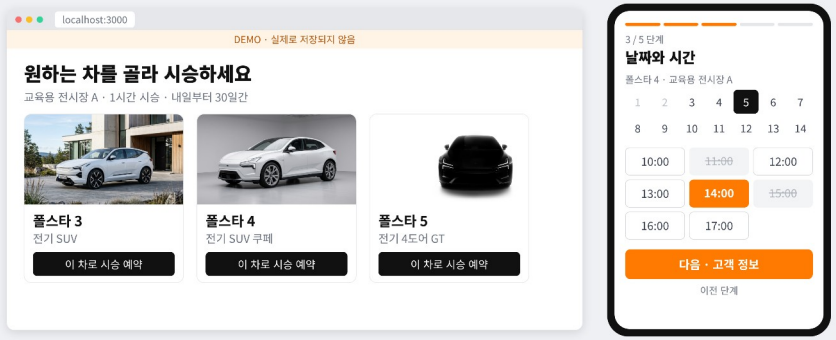
[입력] 프롬프트 · Claude 앱 · Code 탭 입력창

```
plan.md의 2단계를 진행하세요.
현재 폴더에 Next.js(App Router, TypeScript,
Tailwind) 프로젝트를 만들되 docs와 data는 보존하세요.
DB 없이 data 폴더의 CSV와 JSON을 mock으로 씁니다.

화면은 requirements.md 3장을 따르세요.
assets/images를 public/images로 복사하고
models.csv의 image_file로 차종 카드에 연결하세요.
상단에 'DEMO · 실제로 저장되지 않음' 띠를 붙이세요.

끝나면 개발 서버를 켜고 앱의 미리보기 브라우저에
띄워 주세요. 여기서 멈추세요.
```

[출력] 결과 · 브라우저 localhost:3000



피드백 · 이것을 확인한다

- 사진이 맞는 카드에 붙었나
- 다섯 단계를 끝까지 가는가

직접 눌러 보고 증상을 문장으로 돌려준다

고칠 점은 '무엇을 했더니 무엇이 됐다'의 형태로 쓴다

누구 수강생 → Claude Code

어디 미리보기 → Code 탭

눈으로 확인할 것

- 필수 항목이 비면 다음으로 안 넘어가는가
- 뒤로 갔다 와도 선택이 남는가
- 어제, 오늘, 31일 뒤는 못 고르는가
- 휴대폰 형식이 틀리면 막히는가
- 마케팅 동의가 꺼진 채 시작하는가

[입력] 프롬프트 · Code 탭 · 모바일 점검

```
Playwright로 360px 폭에서 / 와 /reserve의
다섯 단계를 차례로 진행하며 스크린샷을 찍어
docs/screens에 저장하고 열어 주세요.
가로 스크롤이나 잘린 글자가 있으면 고치세요.
```

[입력] 프롬프트 · Code 탭 · 문제 고치기 예시

```
뒤로 갔다가 돌아오면 선택한 날짜가 사라집니다.
선택값이 유지되게 고치고, 같은 흐름을 직접
다시 실행해 결과를 알려 주세요.
```

03

3~5단계 · 데이터

DB를 만들고 예약을 실제로 저장한다

키는 편집기에만 넣고 대화창에는 절대 넣지 않는다

Supabase 가입만은 사람이 한다

가입을 마치면 프로젝트 생성부터 키 기록까지 Claude Code가 이어받는다

1
supabase.com
오른쪽 위
Start your project

→

2
로그인 방식
Continue with
GitHub 또는 이메일

→

3
조직 만들기
이름을 정하고
Personal과 Free를
고른다

→

4
여기서 멈춤
새 프로젝트 화면이
나오면 닫는다
(이후는 Claude 몫)

약관 동의, 이메일 인증, 요금제 선택은 본인 확인이 필요해 Claude가 대신할 수 없다.
프로젝트 생성, 키 조회, DB 적용은 로그인만 되어 있으면 Claude가 한다.

가입은 네 화면이면 끝난다

GitHub 계정이 있으면 가장 빠르다. 버튼 이름은 바뀔 수 있다

누구 수강생 (직접)

어디 브라우저 supabase.com

[입력] 직접 하는 일

① 브라우저에서 supabase.com에 접속해 오른쪽 위 Start your project를 누른다

② Continue with GitHub를 누르고 GitHub 화면에서 Authorize를 누른다. GitHub 계정이 없으면 이메일로 가입하고, 받은 메일의 확인 링크를 누른다

③ 조직(Organization) 만들기 화면에서 이름은 aju-edu, 유형은 Personal, 요금제는 Free를 고른다

④ 새 프로젝트 만들기 화면이 나오면 아무것도 넣지 않고 닫는다. 프로젝트는 Claude Code가 만든다

[출력] 결과

```
Supabase 계정 1개
조직 aju-edu (Free)
프로젝트 0개
```

피드백 · 이것을 확인한다

- Free 요금제를 골랐나
- 결제 정보는 넣지 않는다
- 프로젝트는 만들지 않는다

CLI 로그인은 명령 한 줄과 승인 한 번이다

Claude Code가 내 Supabase 계정으로 일하려면 이 PC에서 한 번 로그인해야 한다

누구 수강생 (명령 한 줄)

어디 Code 탭의 터미널 창

[입력] 명령어 · Code 탭 터미널 창 (Ctrl+` 로 연다)

```
# 1) Code 탭에서 Ctrl+` 로 터미널 창을 연다
npx supabase login

# 2) 설치 여부를 물으면 y, 이어서 Enter
# 3) 브라우저가 열리면 로그인 상태에서 승인
# 4) 확인 코드가 나오면 터미널에 입력하고 Enter
```

[출력] 결과 · 터미널 창

```
브라우저 승인 안내 문구
토큰 생성 완료 문구
You are now logged in.
```

피드백 · 이것을 확인한다

- logged in 문구가 보이나
- 로그인은 이 PC에만 남는다
- 창을 닫고 Code 탭으로

프로젝트 생성과 키 기록은 Claude Code가 한다

키는 사람 손을 거치지 않고 .env.local로 바로 들어간다

누구 Claude Code (붙여넣기) **어디** Claude 앱 Code 탭

[입력] 프롬프트 · Claude 앱 · Code 탭 입력창

```
Supabase CLI(npx supabase)로 개발용 프로젝트를
만들고 이 앱에 연결하세요.
- orgs list로 내 조직 ID를 확인하세요
- 프로젝트 이름 aju-testdrive-dev, 리전 서울
  (ap-northeast-2)
- DB 비밀번호는 강력하게 생성해 .env.local에만 저장
- 프로젝트가 준비되면 API 키를 조회해 .env.local에
  URL, publishable key, secret key, APP_URL을 기록
- 키와 비밀번호는 대화에 절대 출력하지 마세요

공식 Next.js SSR 문서 방식대로 브라우저용과 서버용
클라이언트를 나누고, secret key 모듈은 server-only로.
.env.example과 .gitignore를 만들고 .env.local이
Git에서 빠졌는지 확인하세요.
끝나면 대시보드 주소와 채워진 항목 이름만 알려 주세요.
```

[출력] Claude 응답

```
조직 aju-edu 확인
aju-testdrive-dev 생성 (서울)
.env.local 항목 5개 채움
  (값은 표시하지 않음)
대시보드 주소 안내
```

피드백 · 이것을 확인한다

- 응답에 키 문자열이 없나
- 대시보드에 프로젝트가 있나

예약 데이터는 테이블 여덟 개로 나눈다

차량과 시작시각의 조합이 슬롯이고, 예약은 슬롯 하나를 가리킨다



slot 1개에 pending·confirmed 예약은 최대 1건 — partial unique index가 DB에서 강제한다

동시에 눌러도 예약은 한 건만 들어간다

판정은 화면이 아니라 DB 트랜잭션과 partial unique index가 한다



SQL 작성부터 적용까지 Claude Code가 한다

Supabase CLI가 .env.local의 토큰으로 개발 DB에 접속한다

누구 Claude Code (붙여넣기)　**어디** Claude 앱 Code 탭

[입력] 프롬프트 · Claude 앱 · Code 탭 입력창

```
requirements.md 4장과 5장을 따라
supabase/migrations에 SQL을 만드세요.
seed는 data 폴더의 CSV로 만들고, 다시 실행해도
중복되지 않게 하세요.

적용 전에 SQL의 보안과 중복 예약 방어를
스스로 검토하고 결과를 짧게 보고하세요.

.env.local의 DB_PASSWORD를 값 출력 없이 써서
Supabase CLI로 개발 DB에 연결(link)하고 적용하세요.
적용 후 테이블별 행 수를 표로 보여 주세요.
테이블을 지우는 SQL로 오류를 해결하지 마세요.
```

[출력] 결과 · 테이블별 행 수

```
models              3
locations           1
vehicles            3
slots             720
나머지 4개 테이블    0
```

피드백 · 이것을 확인한다

- Table Editor로 slots 확인
- 오류는 '4 피드백' 프롬프트로
- CLI가 막히면 순서를 묻는다

고객 화면을 실제 DB에 연결한다

이 단계부터 예약이 남는다. 테스트 고객 01번으로 확인한다

누구 Claude Code (붙여넣기)　**어디** Claude 앱 Code 탭

[입력] 프롬프트 · Claude 앱 · Code 탭 입력창

```
고객 예약 화면을 Supabase에 연결하세요.
requirements.md 6장의 규칙을 모두 지키세요.
- 슬롯 조회와 예약 생성은 서버 API로만 처리
- 예약 생성은 4단계의 DB 트랜잭션 함수를 호출
- 중복 클릭 방지와 idempotency key 적용
- DB 장애 때 성공 화면을 보이지 않음
- rate limit은 rate_limits 테이블로 구현
- 동의 문구는 docs/privacy_consent_draft.md 사용

DEMO 띠는 개발 DB 연결이 확인되면 떼세요.
끝나면 test_customers.csv의 01번 고객으로 직접
예약 1건을 넣고, 저장된 행을 이름과 번호를
가린 채 보여 주세요.
```

[출력] 결과 · 저장된 1행 (예시)

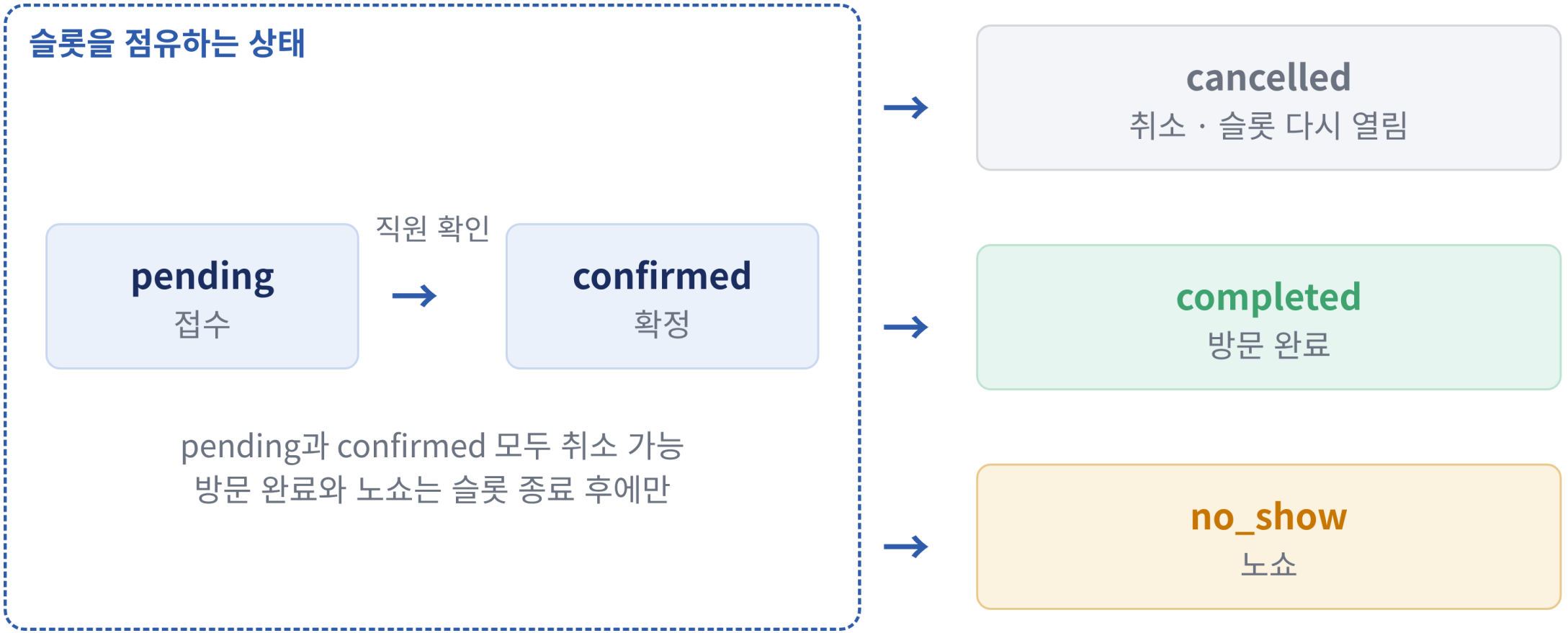
예약번호	R-7Q2K
고객	테*고객01
휴대폰	010-****-0001
상태	pending
일정	폴스타 4, 14:00
동의 버전	edu-draft-2026-10

피드백 · 이것을 확인한다

- 새로고침해도 남는가
- 그 시간이 막혔는가
- 주소창에 개인정보가 없나

상태 다섯 개가 슬롯을 잡거나 놓는다

허용되는 전이는 data/status_transitions.csv에 정리돼 있다



04

6~7단계 · 관리와 검증

직원 화면을 만들고 일부러 깨뜨려 본다

권한 검사와 동시 요청을 자동 테스트로 확인한다

직원 계정까지 Claude Code가 만든다

임시 비밀번호는 내 PC의 파일에만 남고, 권한은 서버와 DB가 검사한다

누구 Claude Code (붙여넣기)　**어디** Claude 앱 Code 탭

[입력] 프롬프트 · Claude 앱 · Code 탭 입력창

```
관리자 로그인과 /admin 화면을 만드세요.
requirements.md 7장을 따르고, 공개 가입 화면은
만들지 마세요.

직원 계정 staff@example.com을 Supabase Auth 관리
API로 만들고(이메일 확인 완료 상태), 그 UUID를
admin_users에 등록하세요.
임시 비밀번호는 무작위로 만들어 staff-login.local.txt
에만 저장하고 Git에서 빼세요. 대화에 출력하지 마세요.

끝나면 로그아웃 상태와 일반 계정으로 /admin과
관리자 API를 직접 호출해 거절되는지 보여 주고,
staff-login.local.txt를 메모장으로 열어 주세요.
```

[출력] 결과 · 권한 점검

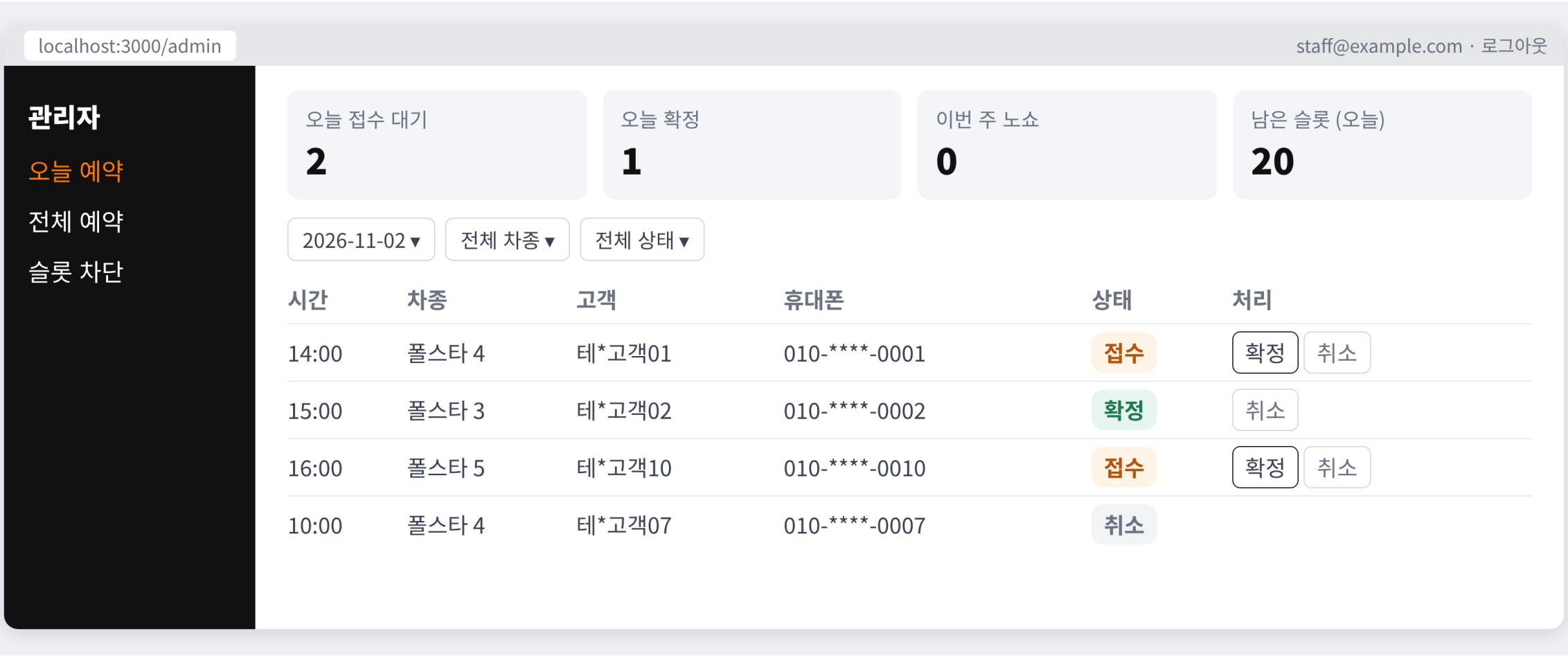
```
staff@example.com 생성
admin_users 1행 등록
로그아웃 API   → 401
일반 계정 API  → 403
메모장에 임시 비밀번호
```

피드백 · 이것을 확인한다

- 그 비밀번호로 로그인한다
- 확정과 취소를 해 본다
- 목록 번호가 가려졌나

직원은 오늘 예약을 한 화면에서 처리한다

목록은 이름과 번호를 가리고, 접수 건에만 확정 버튼이 보인다 (화면 예시)



테스트 14개로 일부러 시스템을 깨뜨린다

시나리오는 docs/test_scenarios.md에, 결과는 docs/test-report.md에 있다

누구 Claude Code (붙여넣기)

어디 Claude 앱 Code 탭

[입력] 프롬프트 · Claude 앱 · Code 탭 입력창

```
docs/test_scenarios.md의 14개 시나리오를
자동 테스트로 작성하고 실행하세요.
개발 DB의 테스트 데이터만 쓰고, 끝나면 지우세요.
브라우저 흐름은 Playwright로 확인하세요.

lint, typecheck, build도 실행하세요.
결과는 docs/test-report.md에 통과, 실패, 미실행으로
기록하세요. 실패는 고친 뒤 그 항목만 다시 실행하고,
미실행을 통과로 적지 마세요.
```

[출력] Claude가 생성 · test-report.md

```
| #  | 시나리오        | 결과 |
| 1  | 동시 요청       | 통과 |
| 2  | 같은 키 재전송  | 통과 |
| 8  | 공개 키 조회    | 통과 |
| 13 | rate limit      | 수정 후 통과 |
lint, typecheck, build 결과
```

피드백 · 이것을 확인한다

- 미실행이 통과로 적혔나
- 실패는 고친 뒤 재실행했나
- 두 창으로 동시 제출해 본다

05

8~9단계 · 배포

인터넷 주소를 받고 기존 사이트에 잇는다

로그인 명령 한 줄과 승인만 사람이 하고 나머지는 Claude Code가 한다

코드는 GitHub로, 화면은 Vercel로 나간다

비밀값은 Git에 올리지 않고 Vercel 환경변수로만 전달한다

내 PC
localhost:3000

→

GitHub
비공개 저장소

→

Vercel 미리보기
개발 DB에 연결

→

운영 배포
운영 DB 별도

↓

기존 아주 사이트
'시승 예약' 버튼 연결

비밀값이 가는 곳

앱용 변수 4개는 Vercel 환경변수로 등록한다
DB 비밀번호와 직원 임시 비밀번호는 내 PC에만 둔다

미리보기와 운영은 Supabase 프로젝트를 나눈다 — 테스트 예약이 운영 DB에 섞이지 않는다

올리기 전에 비밀값부터 검사한다

저장소는 비공개로 만든다. 로그인 명령만 터미널 창에 직접 친다

누구 Claude Code (붙여넣기)

어디 Code 탭과 터미널 창

[입력] 프롬프트 · Claude 앱 · Code 탭 입력창

```
이 프로젝트를 GitHub 비공개 저장소
aju-testdrive에 올리세요.
먼저 .env.local, 키, 고객정보, DB 덤프가 Git에
추적되지 않는지 검사하세요.
gh CLI가 없으면 설치하세요. 로그인이 필요하면
앱의 터미널 창에 내가 칠 명령 한 줄만 알려 주세요.
커밋하고 push한 뒤 저장소 주소를 알려 주세요.
```

[출력] 결과

```
검사: .env.local 추적 안 됨
검사: 키 문자열 0건
gh repo create aju-testdrive --private
push 완료, 저장소 주소 안내
```

피드백 · 이것을 확인한다

- 웹에 .env.local이 없나
- Private 표시가 있나
- 로그인은 터미널 창에서

미리보기 주소에서 두 흐름을 다시 본다

미리보기는 개발 DB에, 운영은 별도 Supabase 프로젝트에 연결한다

누구 Claude Code (붙여넣기)　**어디** Code 탭과 터미널 창

[입력] 프롬프트 · Claude 앱 · Code 탭 입력창

```
npx vercel로 미리보기 배포를 하세요.
로그인이 필요하면 앱의 터미널 창에 내가 칠
명령 한 줄만 알려 주세요.
.env.local의 앱용 변수 4개만 값 출력 없이 Vercel에
등록하세요. DB 비밀번호와 staff-login 파일은
올리지 마세요. 미리보기는 개발 DB에 연결합니다.

배포 주소가 나오면 Supabase Auth의 Site URL과
Redirect URL을 CLI로 반영하고, 안 되면 대시보드에서
넣을 위치와 값을 알려 주세요.
배포 주소에서 예약 1건과 관리자 로그인을
Playwright로 확인하고 결과를 보여 주세요.
```

[출력] 결과

```
Preview 주소
  https://aju-testdrive-...
  .vercel.app
환경변수 4개 등록 (값 숨김)
Site URL과 Redirect URL 안내
```

피드백 · 이것을 확인한다

- Auth URL이 반영됐나
- 휴대폰으로 접속해 본다

기존 사이트는 버튼만 예약 앱으로 바꾼다

기존 사이트 원본 프로젝트가 있을 때 그 폴더에서 새 세션으로 한다

누구 Claude Code (붙여넣기)　**어디** 원본 폴더의 Code 탭

[입력] 프롬프트 · Code 탭 · 원본 프로젝트 폴더

```
(기존 사이트 원본 폴더를 Code 탭에서 열고 실행)
이 프로젝트의 프레임워크와 라우팅부터 확인하세요.
'시승 예약' 버튼과 모바일 메뉴를 아래 주소로
연결하세요: [8단계에서 확인한 배포 주소]
현재 화면 스타일을 유지하고, 상대주소 /reserve로
임의 연결하지 마세요.
바뀐 파일과 확인 방법을 알려 주세요.
```

[출력] 결과 (예시)

```
바뀐 파일 2개
  헤더의 시승 예약 버튼
  모바일 메뉴
```

피드백 · 이것을 확인한다

- 버튼이 예약 앱을 여는가
- 모바일 메뉴도 확인한다
- 원본이 없으면 담당자에게 주소를 넘긴다

막히면 증상을 붙여 넣고 원인부터 묻는다

어느 단계에서든 Code 탭 입력창에 붙여 넣는다. 오류 문구의 키 값은 지운다

[입력] 프롬프트 · 빨간 오류가 났을 때

```
다음 오류를 해결하세요. 원인을 먼저 설명하고
필요한 파일만 고치세요. 패키지 전체 업데이트나
DB 삭제는 하지 마세요. 고친 뒤 같은 명령으로
다시 확인하세요.
[키와 개인정보를 지운 오류 문구]
```

[입력] 프롬프트 · 화면이 예전 그대로일 때

```
지금 켜진 개발 서버의 폴더와 포트를 확인하세요.
내가 보는 주소와 수정한 프로젝트가 같은지
점검하고, 필요하면 서버를 다시 켜 주세요.
```

[입력] 프롬프트 · 예약 저장이 실패할 때

```
예약 저장 실패를 진단하세요. 환경변수는 있는지만
보고 값은 출력하지 마세요. migration, 서버 API,
함수 권한, RLS, 제약조건 순서로 확인하세요.
RLS를 끄거나 anon에 권한을 주지 마세요.
```

[입력] 프롬프트 · 새 세션에서 이어 갈 때

```
CLAUDE.md, docs/plan.md, docs/test-report.md와
Git 상태를 읽고 끝난 단계와 남은 단계를
구분해 요약하세요. 기존 구현을 보존하면서
다음 미완료 단계부터 이어 가세요.
```

열 단계가 끝나면 이 산출물이 손에 남는다

단계마다 Git 커밋이 남으므로 언제든 이전 상태로 돌아갈 수 있다

직접 조작

0

준비

Claude Code 실행

1

규칙

CLAUDE.md

2

화면

localhost 화면

직접 조작

3

Supabase

가입 후 자동 연결

4

DB

테이블 8개

5

저장

예약 1건 저장

6

관리자

확정과 취소

7

테스트

test-report

8

배포

미리보기 주소

9

버튼 연결

기존 사이트

앰버 표시는 앱 설치와 Supabase 가입처럼 사람만 할 수 있는 단계. 나머지는 프롬프트만 붙여 넣는다

완료 점검

운영 전에 열세 가지를 직접 확인한다

같은 목록이 docs/acceptance_checklist.md에 있다

기능과 보안

- 접수와 확정이 구분된다
- 새로고침해도 예약이 남는다
- 동시 요청 중 한 건만 들어간다
- 취소하면 미래 슬롯이 다시 열린다
- 직원만 개인정보를 볼 수 있다
- 비밀키가 브라우저와 Git에 없다
- 서울 시간 기준으로 날짜가 맞는다

운영 준비

- 실제 전시장, 시승차, 휴일을 확인했다
- 동의 문구의 보유기간을 확정했다
- rate limit이 DB로 동작한다
- 접수 확인 담당자를 정했다
- 배포 주소에서 두 흐름을 다시 봤다
- 테스트 실제 결과가 기록돼 있다

도입 효과는 처리 시간과 확정률로 잰다

출시 전과 후의 같은 길이 기간을 비교한다

기존 업무	시스템 적용 후	볼 지표
전화와 메신저로 날짜 조율	고객이 빈 슬롯을 직접 고름	예약 1건당 직원 처리 시간
엑셀에 손으로 입력	예약 DB에 자동 기록	누락과 중복 건수
접수 후 확인이 늦어짐	접수 목록에서 바로 처리	접수에서 확정까지 걸린 시간
결과가 여기저기 흩어짐	방문, 취소, 노쇼가 쌓임	시승 완료율과 노쇼율

알림을 실제로 연결하기 전에는 '알림 자동화' 효과를 성과로 계산하지 않는다

A

부록 · 터미널 사용

터미널에서 Claude Code를 쓰는 경우

데스크톱 앱 대신 PowerShell과 VS Code로 진행할 때의 설치와 실행 방법이다

터미널용 설치는 PowerShell 명령 몇 줄로 끝난다

Windows 기준이다. 데스크톱 앱에서 Git 오류가 날 때도 이 Git 설치 줄만 쓴다

누구 수강생 (직접)

어디 Windows PowerShell

0 준비 · 1 규칙 · 2 화면 · 3 연결 · 4 DB · 5 저장 · 6 관리자 · 7 테스트 · 8 배포 · 9 버튼

[입력] 명령어 · PowerShell에 한 줄씩

```
# 1) 편집기, Git, Node.js
winget install -e --id Microsoft.VisualStudioCode
winget install -e --id Git.Git
winget install -e --id OpenJS.NodeJS.LTS

# 2) Claude Code
irm https://claude.ai/install.ps1 | iex

# 3) PowerShell을 닫았다 다시 열고 확인
claude --version
```

[출력] 결과 · PowerShell

```
설치 진행 표시와 완료 메시지
claude --version 입력 시
  버전 번호 한 줄
```

피드백 · 이것을 확인한다

- 버전 번호가 나오는가
- 못 찾으면 창을 다시 연다
- 유료 계정이 준비됐는가

터미널에서는 VS Code 안에서 claude를 실행한다

이 방식이면 본문의 'Code 탭'을 'VS Code 터미널 A'로 바꿔 읽는다

누구 수강생 (직접)

어디 PowerShell → VS Code

0 준비 · 1 규칙 · 2 화면 · 3 연결 · 4 DB · 5 저장 · 6 관리자 · 7 테스트 · 8 배포 · 9 버튼

[입력] 명령어 · PowerShell, 이어서 VS Code 터미널

```
mkdir aju-testdrive
cd aju-testdrive
code .

# VS Code 메뉴: 터미널 → 새 터미널 (A)
claude

# 터미널 오른쪽 + 버튼으로 터미널 B 추가
```

[출력] 결과 · VS Code

브라우저에서 Claude 로그인 승인
터미널 A에 Claude Code 입력창
여기에 앞으로 프롬프트를 넣는다

피드백 · 이것을 확인한다

- A에 입력창이 보이는가
- code 실패 시 폴더 열기
- 직접 명령은 B에만 친다